T2_SUMMARY_VHA

Task 1: Simple Linear Regression

Perform Simple Linear Regression on the dataset.

Use: test_size = 0.2

random_state = 42

Train the model and calculate:

Mean Squared Error (MSE)

R^2 Score

In [4]:
```python
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score

# 2. Load dataset
data = pd.read_csv('student_scores.csv')

# 3. Define features (X) and target (y)
X = data[['Hours']]      # independent variable
y = data['Scores']       # dependent variable

# 4. Split dataset (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 5. Create model
model = LinearRegression()

# 6. Train model
model.fit(X_train, y_train)

# 7. Predict on test data
y_pred = model.predict(X_test)

# 8. Evaluate model
mse = mean_squared_error(y_test, y_pred)

# 9. Print results
print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print('r2_score',r2_score(y_test,y_pred))

# 10. Example prediction
print("Prediction for 5 hours:", model.predict([[5]]))
```

```
Coefficient (Slope): 9.682078154455697
Intercept: 2.826892353899737
Mean Squared Error: 18.943211722315272
r2_score 0.9678055545167994
Prediction for 5 hours: [51.23728313]
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Task 2: Linear Regression Optimization

Use different:

test_size = [0.05, 0.1, 0.15, 0.2, 0.25]

For each test size:

Train model and calculate R^2 and MSE

Plot:

Test Size vs R^2 Score

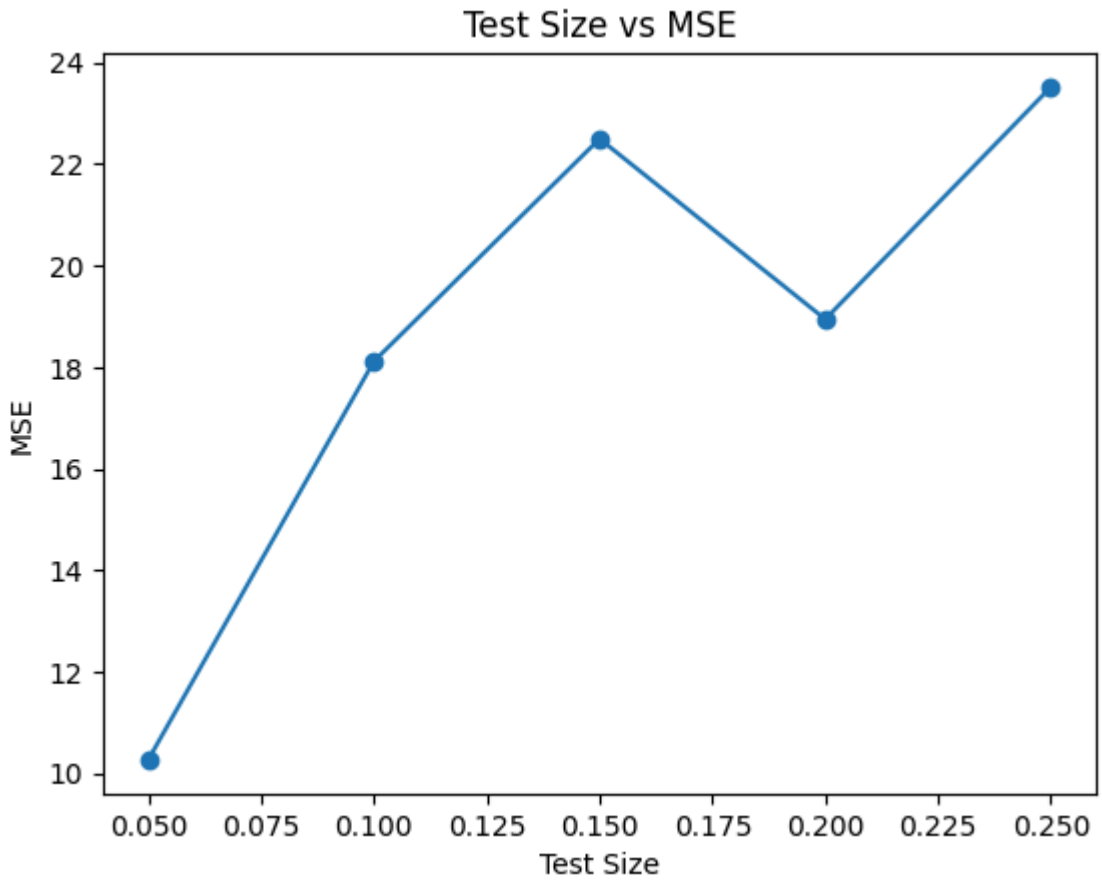
Test Size vs MSE

Select the best model (highest R^2) and predict output for a new value

In [7]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('student_scores.csv')

# 3. Define features (X) and target (y)
X = data[['Hours']]
y = data['Scores']

# 4. Different test sizes
test_sizes = [0.05, 0.1, 0.15, 0.2, 0.25]

r2_list = []
mse_list = []

best_r2 = -1
best_model = None
best_test_size = None

# 5. Loop through test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
```

```
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)

    r2_list.append(r2)
    mse_list.append(mse)

    # Store best model
    if r2 > best_r2:
        best_r2 = r2
        best_model = model
        best_test_size = ts

# 6. Print best model info
print("===== BEST MODEL DETAILS =====")
print("Best Test Size:", best_test_size)
print("Best R2 Score:", best_r2)
print("Coefficient (Slope):", best_model.coef_[0])
print("Intercept:", best_model.intercept_)

# 7. Plot Test Size vs R2 Score
plt.figure()
plt.plot(test_sizes, r2_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("R2 Score")
plt.title("Test Size vs R2 Score")
plt.show()

# 8. Plot Test Size vs MSE
plt.figure()
plt.plot(test_sizes, mse_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("MSE")
plt.title("Test Size vs MSE")
plt.show()

# 9. External Data Prediction (IMPORTANT PART)

# New unseen data
new_hours = [[2], [4.5], [6], [8], [9.5]]

# Predict using best model
new_predictions = best_model.predict(new_hours)

print("\n===== EXTERNAL DATA PREDICTIONS =====")
for h, p in zip(new_hours, new_predictions):
    print(f"Hours: {h[0]} -> Predicted Score: {p:.2f}")
```

```
===== BEST MODEL DETAILS =====
Best Test Size: 0.05
Best R2 Score: 0.9842077430211454
Coefficient (Slope): 9.896039318831958
Intercept: 1.8653694779887289
```




```
===== EXTERNAL DATA PREDICTIONS =====
Hours: 2 -> Predicted Score: 21.66
Hours: 4.5 -> Predicted Score: 46.40
Hours: 6 -> Predicted Score: 61.24
Hours: 8 -> Predicted Score: 81.03
Hours: 9.5 -> Predicted Score: 95.88
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

task 3: Multiple Linear Regression

Apply Multiple Linear Regression on dataset Use:

test_size = 0.2

random_state = 42

Train model and evaluate performance

Predict output (target variable) for one new data record

In [10]:
```
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('insurance.csv')

# 3. Convert categorical data to numeric (important)
data = pd.get_dummies(data, drop_first=True)

# 4. Define features (X) and target (y)
X = data.drop('expenses', axis=1)
y = data['expenses']

# 5. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Create model
model = LinearRegression()

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict on test data
y_pred = model.predict(X_test)

# 9. Evaluate model
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))

# 10. Show coefficients
print("\nCoefficients:")
for name, coef in zip(X.columns, model.coef_):
    print(f"{name}: {coef}")

print("Intercept:", model.intercept_)

# 11. Predict for ONE new data point

# Example input:
# age, bmi, children, sex_male, smoker_yes, region_northwest, region_southeast, region_southwe
new_data = [[30, 25.0, 2, 1, 0, 0, 1, 0]]

prediction = model.predict(new_data)

print("\nPredicted Insurance Charge:", prediction[0])
```

```
R2 Score: 0.7835726930039906
MSE: 33600065.35507782

Coefficients:
age: 256.9559592853921
bmi: 337.2714729278702
children: 425.64137565171177
sex_male: -18.519740709140297
smoker_yes: 23650.312302183447
region_northwest: -370.3135113342786
region_southeast: -658.7123820464831
region_southwest: -809.2298783061185
Intercept: -11936.774427292208

Predicted Insurance Charge: 4377.741803014111
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

complete Multiple Linear Regression program that:

tries different test sizes

finds best R^2 score

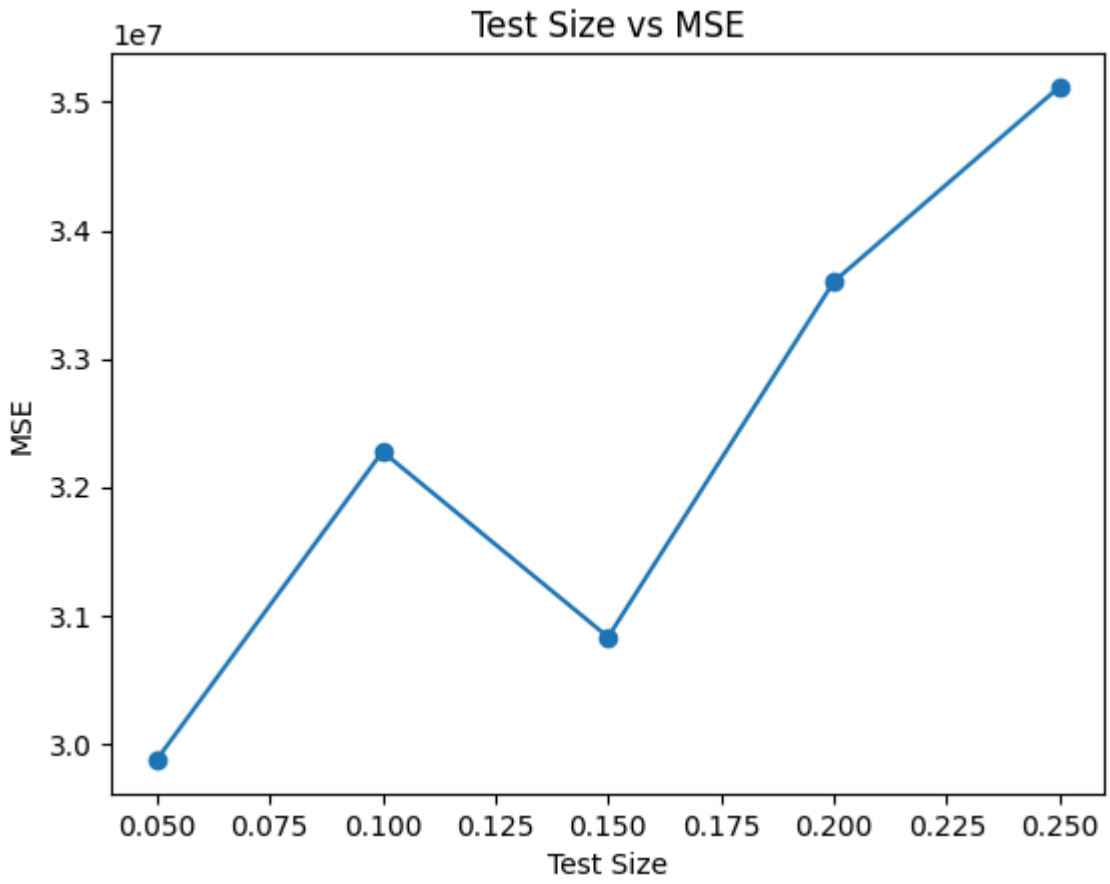
plots Test Size vs R^2 and Test Size vs MSE

predicts one new data point using BEST model

In [12]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('insurance.csv')

# 3. Convert categorical to numeric
data = pd.get_dummies(data, drop_first=True)

# 4. Define X and y
X = data.drop('expenses', axis=1)
y = data['expenses']

# 5. Different test sizes
test_sizes = [0.05, 0.1, 0.15, 0.2, 0.25]

r2_list = []
mse_list = []

best_r2 = -1
best_model = None
best_test_size = None

# 6. Loop for different test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )
```

```
    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)

    r2_list.append(r2)
    mse_list.append(mse)

    # Select best model
    if r2 > best_r2:
        best_r2 = r2
        best_model = model
        best_test_size = ts

# 7. Print best model details
print("===== BEST MODEL =====")
print("Best Test Size:", best_test_size)
print("Best R2 Score:", best_r2)

# 8. Plot Test Size vs R2
plt.figure()
plt.plot(test_sizes, r2_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("R2 Score")
plt.title("Test Size vs R2 Score")
plt.show()

# 9. Plot Test Size vs MSE
plt.figure()
plt.plot(test_sizes, mse_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("MSE")
plt.title("Test Size vs MSE")
plt.show()

# 10. Predict ONE new data point (using best model)

# Format:
# [age, bmi, children, sex_male, smoker_yes, region_northwest, region_southeast, region_southw

new_data = [[35, 27.5, 1, 1, 0, 0, 0, 1]]

prediction = best_model.predict(new_data)

print("\n===== PREDICTION =====")
print("Predicted Insurance Charges:", prediction[0])
```

```
===== BEST MODEL =====
Best Test Size: 0.05
Best R2 Score: 0.8103556402739653
```




```
===== PREDICTION =====
Predicted Insurance Charges: 5827.833735875785
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Here's a clean Polynomial Regression (degree = 2) version for the same insurance.csv dataset using:

test_size = 0.2

random_state = 42

plus prediction for one new data point

In [14]:
```python
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('insurance.csv')

# 3. Convert categorical to numeric
data = pd.get_dummies(data, drop_first=True)

# 4. Define X and y
X = data.drop('expenses', axis=1)
y = data['expenses']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Apply Polynomial Features (degree = 2)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# 7. Train model
model = LinearRegression()
model.fit(X_train_poly, y_train)

# 8. Predict on test data
y_pred = model.predict(X_test_poly)

# 9. Evaluate model
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))

# 10. Predict ONE new data point

# Format:
# [age, bmi, children, sex_male, smoker_yes, region_northwest, region_southeast, region_southw

new_data = [[35, 27.5, 1, 1, 0, 0, 0, 1]]

# Convert to polynomial form
new_data_poly = poly.transform(new_data)

prediction = model.predict(new_data_poly)

print("\nPredicted Insurance Charges:", prediction[0])
```

```
R2 Score: 0.866653268203715
MSE: 20701911.258008774

Predicted Insurance Charges: 5600.486399229701
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
```

tests multiple polynomial degrees

finds best R^2 score

plots Degree vs R^2 and Degree vs MSE

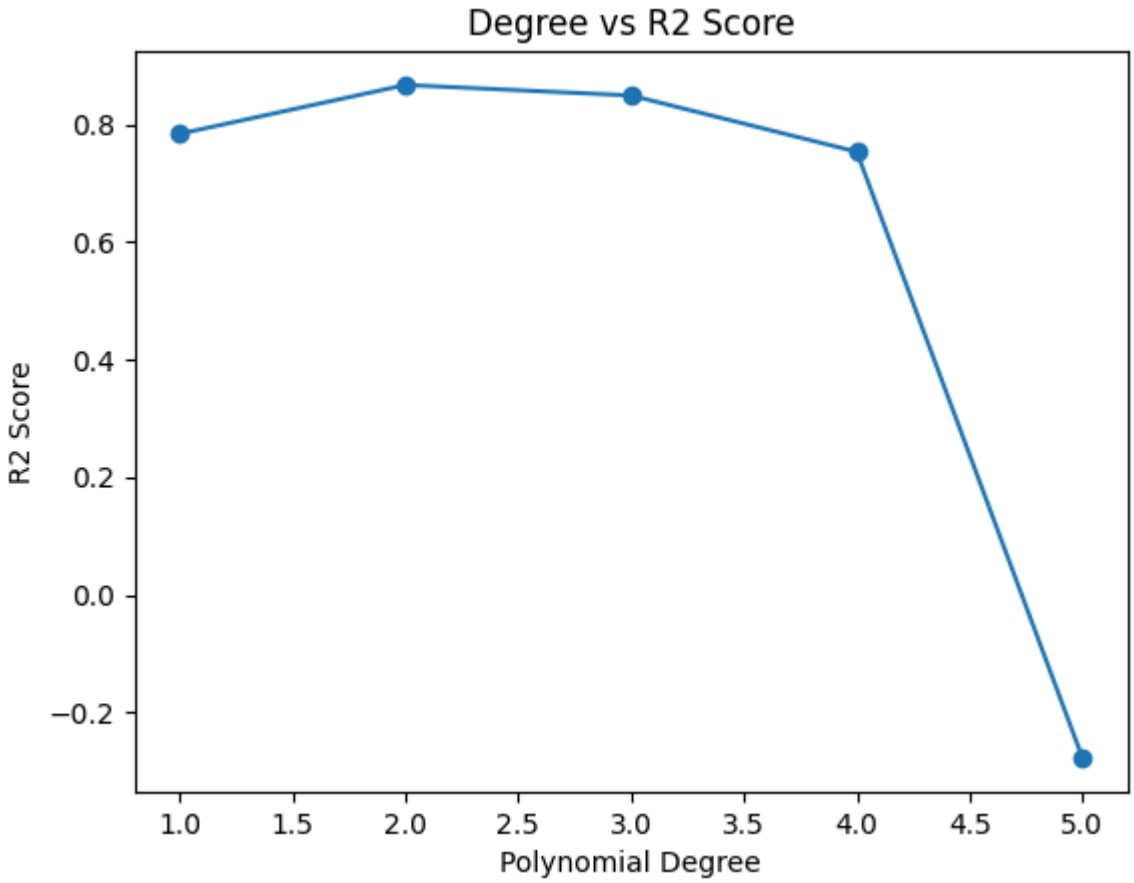
predicts one new data point using BEST model

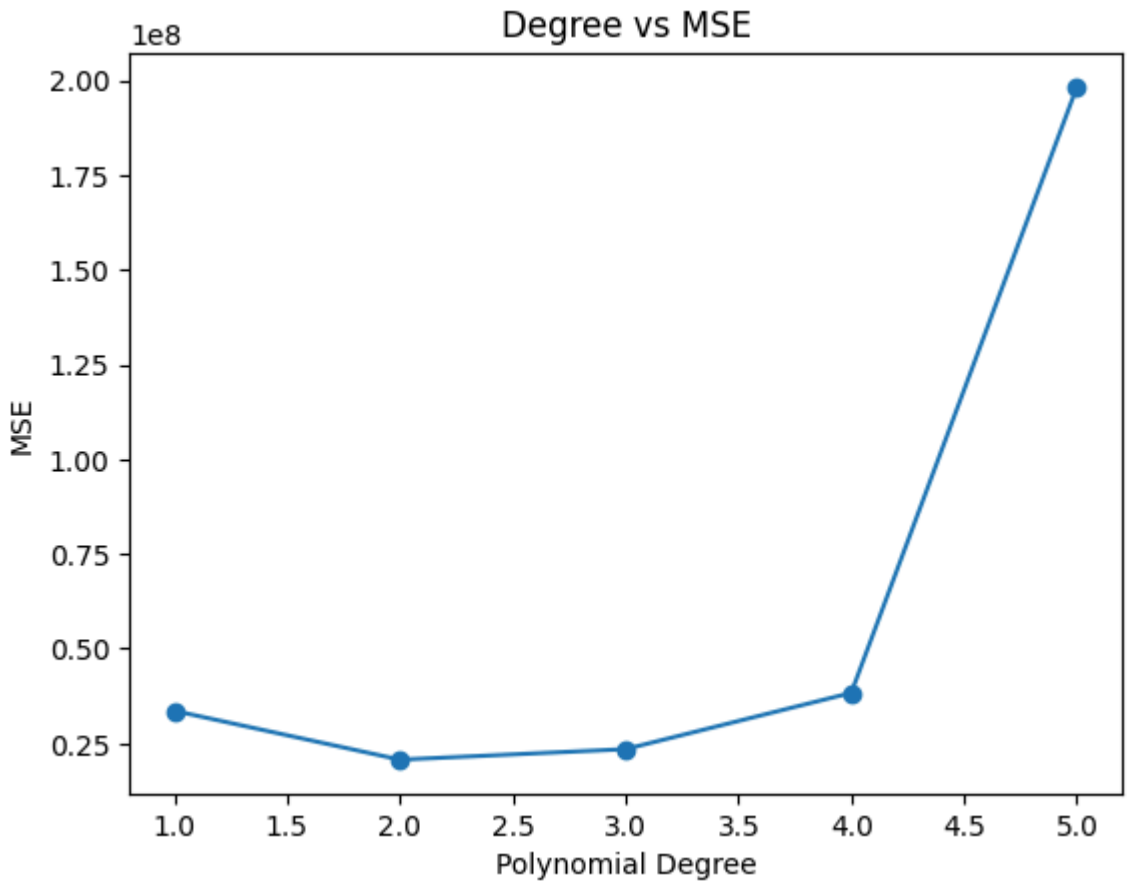
In [16]:

```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('insurance.csv')

# 3. Convert categorical to numeric
data = pd.get_dummies(data, drop_first=True)

# 4. Define X and y
X = data.drop('expenses', axis=1)
y = data['expenses']

# 5. Train-test split (fixed)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Try different polynomial degrees
degrees = [1, 2, 3, 4, 5]

r2_list = []
mse_list = []

best_r2 = -1
best_degree = None
best_model = None
best_poly = None

# 7. Loop over degrees
for d in degrees:
    poly = PolynomialFeatures(degree=d)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)

    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)

    r2_list.append(r2)
    mse_list.append(mse)
```

```
    # Select best model
    if r2 > best_r2:
        best_r2 = r2
        best_degree = d
        best_model = model
        best_poly = poly

# 8. Print best result
print("===== BEST MODEL =====")
print("Best Degree:", best_degree)
print("Best R2 Score:", best_r2)

# 9. Plot Degree vs R2
plt.figure()
plt.plot(degrees, r2_list, marker='o')
plt.xlabel("Polynomial Degree")
plt.ylabel("R2 Score")
plt.title("Degree vs R2 Score")
plt.show()

# 10. Plot Degree vs MSE
plt.figure()
plt.plot(degrees, mse_list, marker='o')
plt.xlabel("Polynomial Degree")
plt.ylabel("MSE")
plt.title("Degree vs MSE")
plt.show()

# 11. Predict ONE new data point using BEST model

# Format:
# [age, bmi, children, sex_male, smoker_yes, region_northwest, region_southeast, region_southw

new_data = [[40, 28.0, 2, 1, 0, 1, 0, 0]]

# Convert using BEST polynomial transformer
new_data_poly = best_poly.transform(new_data)

prediction = best_model.predict(new_data_poly)

print("\n===== PREDICTION =====")
print("Predicted Insurance Charges:", prediction[0])
```

```
===== BEST MODEL =====
Best Degree: 2
Best R2 Score: 0.866653268203715
```






```
===== PREDICTION =====
Predicted Insurance Charges: 8836.953659434073
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
```

try different polynomial degrees + different test sizes

pick the best R^2 overall

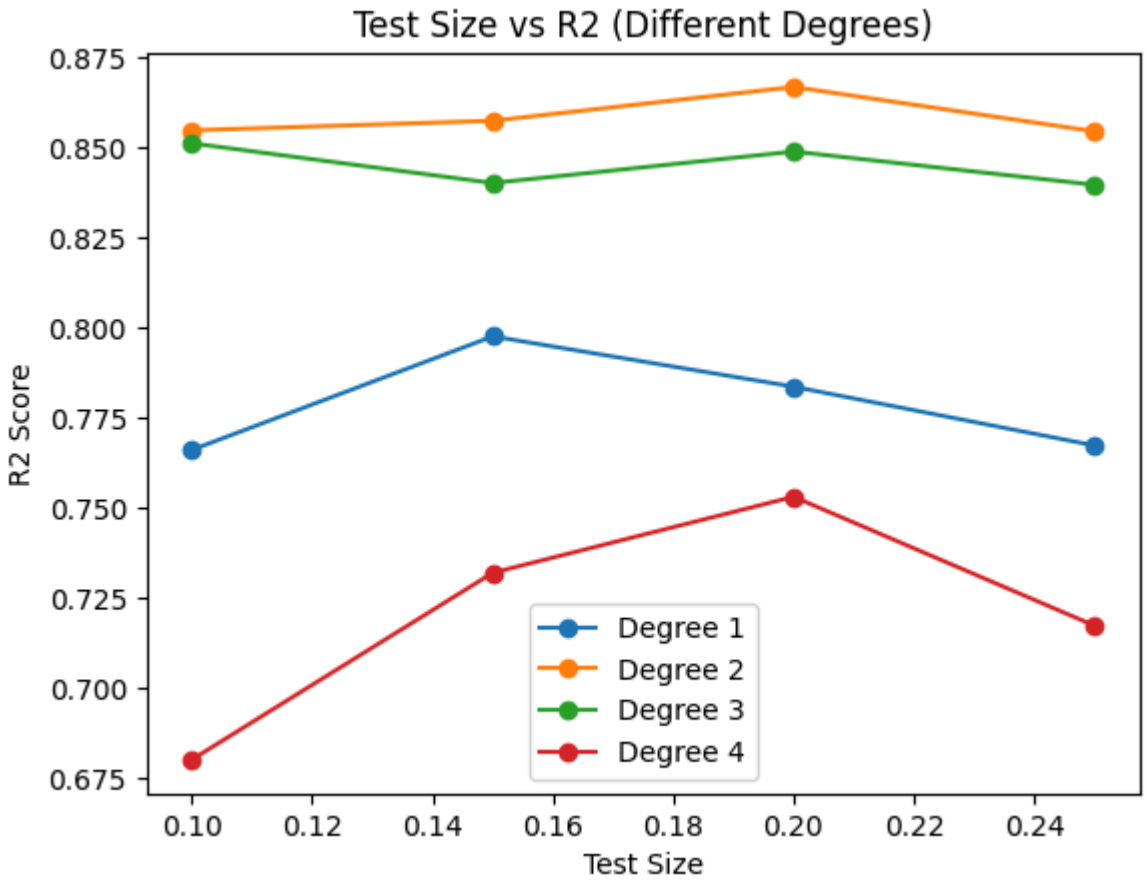
plot results

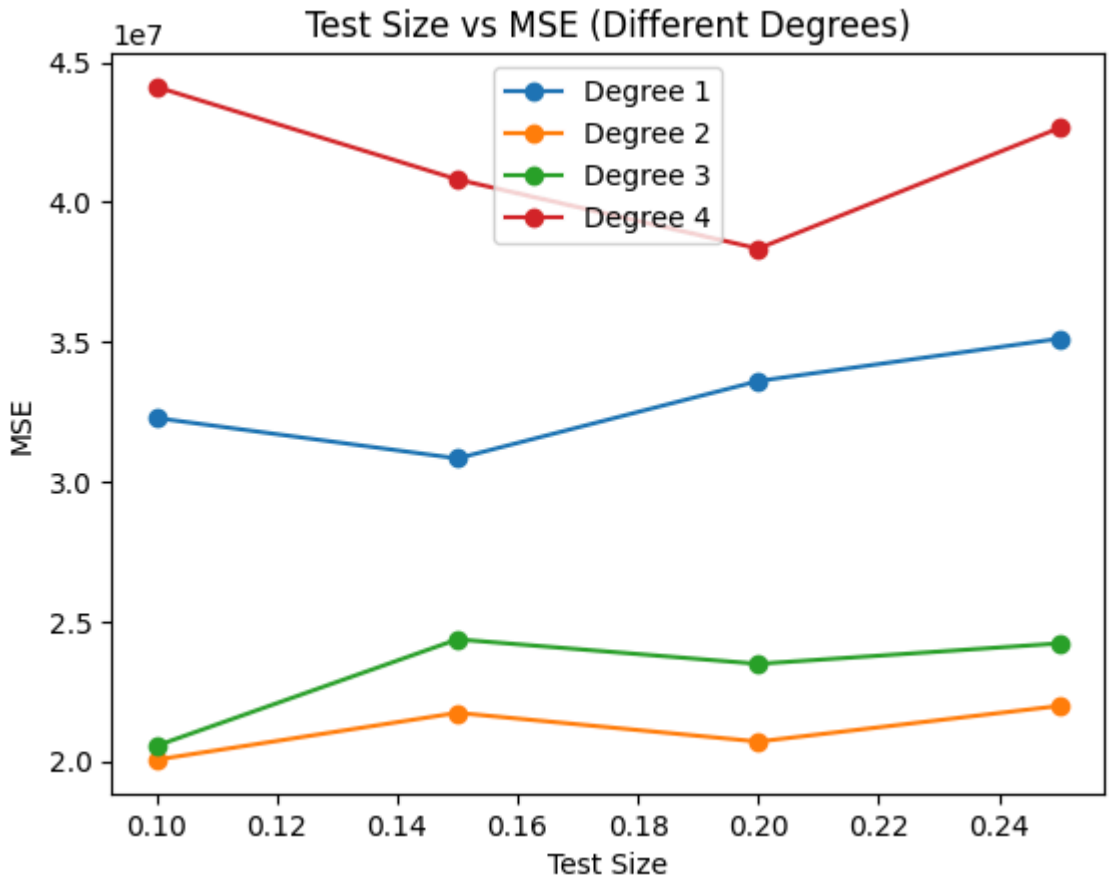
predict one value using the best model

In [18]:
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = pd.read_csv('insurance.csv')

# 3. Convert categorical to numeric
data = pd.get_dummies(data, drop_first=True)

# 4. Define X and y
X = data.drop('expenses', axis=1)
y = data['expenses']

# 5. Define test sizes and degrees
test_sizes = [0.1, 0.15, 0.2, 0.25]
degrees = [1, 2, 3, 4]

results = []

best_r2 = -1
best_model = None
best_poly = None
best_test_size = None
best_degree = None

# 6. Loop through test sizes and degrees
for ts in test_sizes:
    for d in degrees:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=ts, random_state=42
        )

        poly = PolynomialFeatures(degree=d)
        X_train_poly = poly.fit_transform(X_train)
        X_test_poly = poly.transform(X_test)

        model = LinearRegression()
        model.fit(X_train_poly, y_train)

        y_pred = model.predict(X_test_poly)

        r2 = r2_score(y_test, y_pred)
        mse = mean_squared_error(y_test, y_pred)

        results.append((ts, d, r2, mse))

        # Select best model
        if r2 > best_r2:
            best_r2 = r2
            best_model = model
            best_poly = poly
            best_test_size = ts
            best_degree = d
```

```python
# 7. Print best result
print("===== BEST MODEL =====")
print("Best Test Size:", best_test_size)
print("Best Degree:", best_degree)
print("Best R2 Score:", best_r2)

# 8. Convert results to DataFrame for plotting
results_df = pd.DataFrame(results, columns=['Test Size', 'Degree', 'R2', 'MSE'])

# 9. Plot R2 vs Test Size (for each degree)
plt.figure()
for d in degrees:
    subset = results_df[results_df['Degree'] == d]
    plt.plot(subset['Test Size'], subset['R2'], marker='o', label=f'Degree {d}')

plt.xlabel("Test Size")
plt.ylabel("R2 Score")
plt.title("Test Size vs R2 (Different Degrees)")
plt.legend()
plt.show()

# 10. Plot MSE vs Test Size (for each degree)
plt.figure()
for d in degrees:
    subset = results_df[results_df['Degree'] == d]
    plt.plot(subset['Test Size'], subset['MSE'], marker='o', label=f'Degree {d}')

plt.xlabel("Test Size")
plt.ylabel("MSE")
plt.title("Test Size vs MSE (Different Degrees)")
plt.legend()
plt.show()

# 11. Predict ONE new data point using BEST model

# Format:
# [age, bmi, children, sex_male, smoker_yes, region_northwest, region_southeast, region_southw

new_data = [[45, 26.5, 2, 1, 1, 0, 0, 0]]

new_data_poly = best_poly.transform(new_data)

prediction = best_model.predict(new_data_poly)

print("\n===== PREDICTION =====")
print("Predicted Insurance Charges:", prediction[0])
```

```
===== BEST MODEL =====
Best Test Size: 0.2
Best Degree: 2
Best R2 Score: 0.866653268203715
```






```
===== PREDICTION =====
Predicted Insurance Charges: 27393.257937172508
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
```


simple KNN (K-Nearest Neighbors) classification example using:

n_neighbors = 3

test_size = 0.2

random_state = 42

dataset: diabetes.csv

In [1]:
```python
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values manually
# In this dataset, 0 means missing for some columns
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Create model
model = KNeighborsClassifier(n_neighbors=3)

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# 10. Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
specificity = TN / (TN + FP)

# 11. Output
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall (Sensitivity):", recall)
print("Specificity:", specificity)
print("Confusion Matrix:\n", cm)

# 12. Predict ONE new data point

# Format:
# [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction,
```

```
new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = model.predict(new_data)

print("\nPrediction (0=No Diabetes, 1=Diabetes):", prediction[0])
```

```
Accuracy: 0.6582278481012658
Precision: 0.5
Recall (Sensitivity): 0.4444444444444444
Specificity: 0.7692307692307693
Confusion Matrix:
 [[40 12]
 [15 12]]

Prediction (0=No Diabetes, 1=Diabetes): 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

try different neighbor values (K)

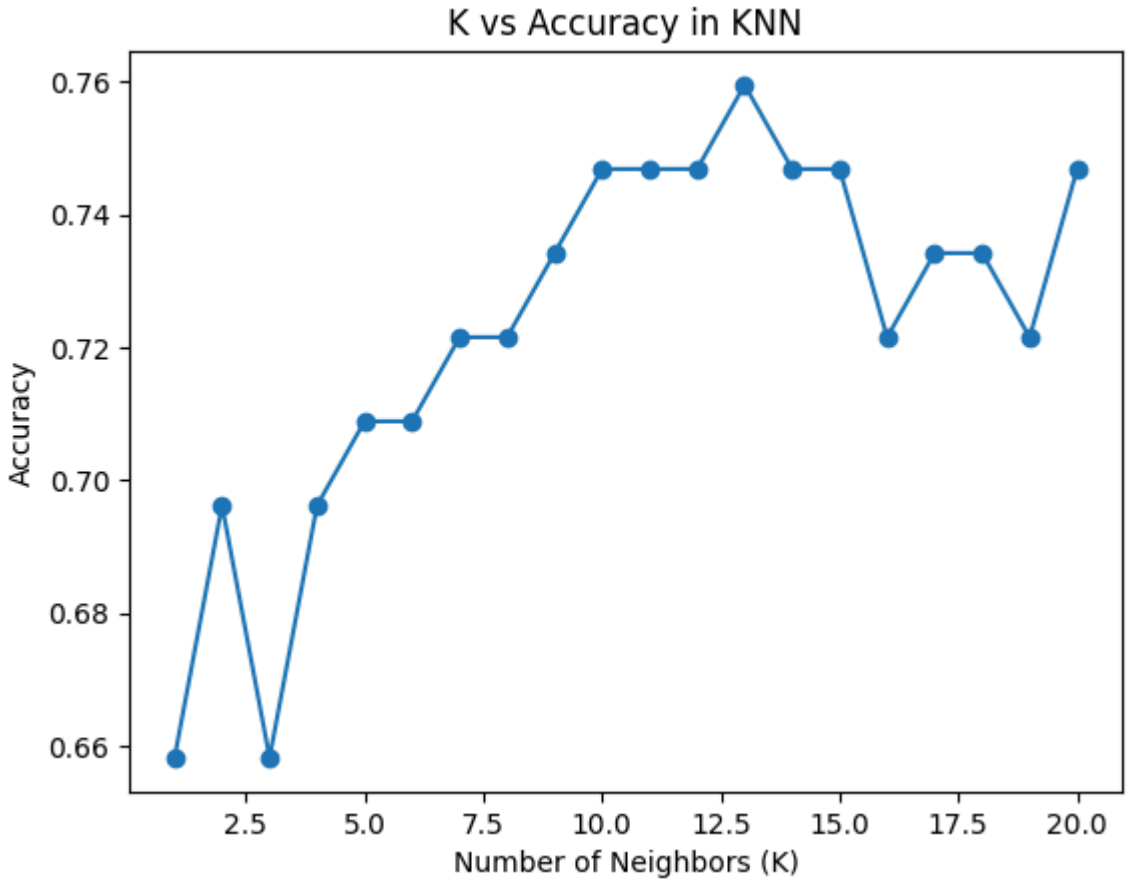
find best accuracy

optionally plot results

In [2]:
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Try different K values
k_values = range(1, 21)

accuracy_list = []

best_k = None
best_accuracy = 0

# 7. Loop for K
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    accuracy_list.append(acc)

    if acc > best_accuracy:
        best_accuracy = acc
        best_k = k

# 8. Print best result
print("Best K:", best_k)
print("Best Accuracy:", best_accuracy)

# 9. Plot K vs Accuracy
plt.figure()
plt.plot(k_values, accuracy_list, marker='o')
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy in KNN")
plt.show()

# 10. Predict using best K
best_model = KNeighborsClassifier(n_neighbors=best_k)
best_model.fit(X_train, y_train)

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best K:", prediction[0])
```

```
Best K: 13
Best Accuracy: 0.759493670886076
```




```
Prediction using best K: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

K fixed (e.g., K = 3)

Try different test sizes

Find best accuracy

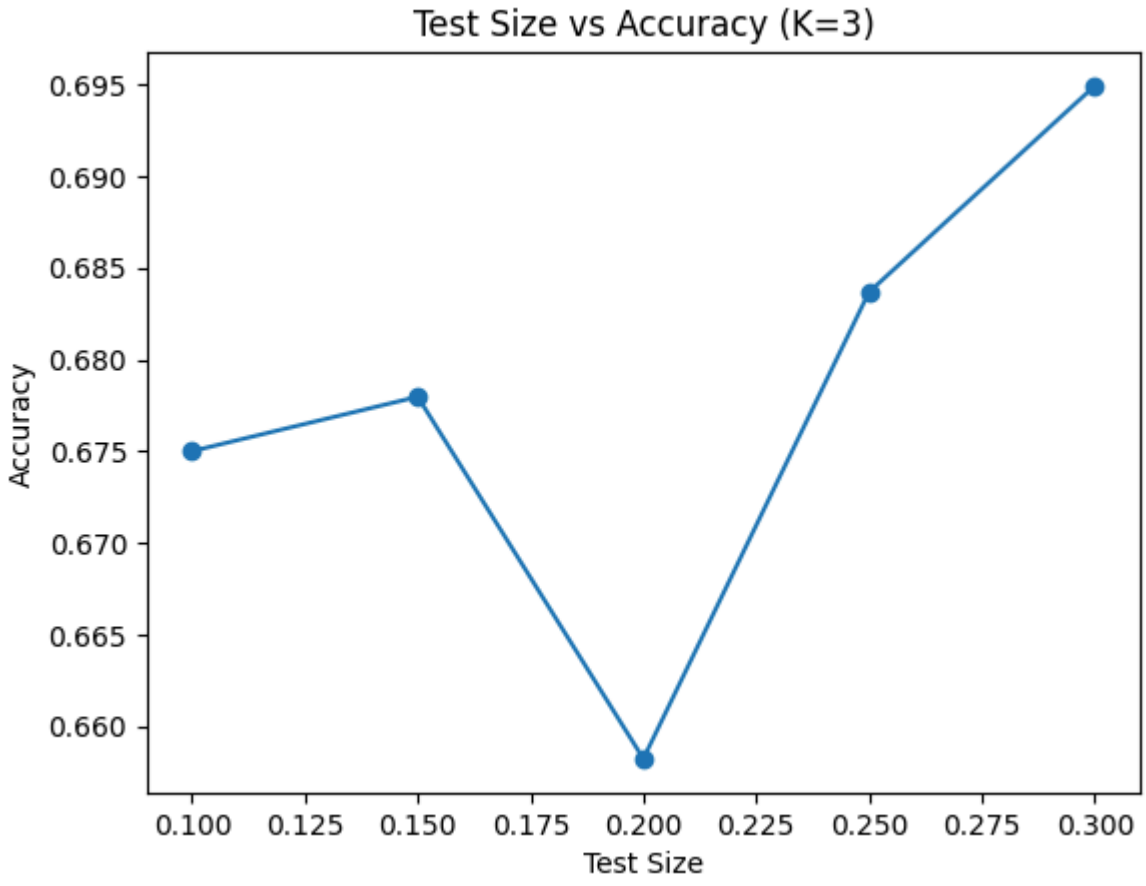
Plot Test Size vs Accuracy

Predict one value using best model

In [3]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Fixed K
k = 3

# 6. Different test sizes
test_sizes = [0.1, 0.15, 0.2, 0.25, 0.3]

accuracy_list = []

best_accuracy = 0
best_test_size = None
best_model = None

# 7. Loop through test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    accuracy_list.append(acc)

    # Store best model
    if acc > best_accuracy:
        best_accuracy = acc
        best_test_size = ts
        best_model = model
```

```
# 8. Print best result
print("Fixed K:", k)
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 9. Plot Test Size vs Accuracy
plt.figure()
plt.plot(test_sizes, accuracy_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("Accuracy")
plt.title("Test Size vs Accuracy (K=3)")
plt.show()

# 10. Predict one new data point using best model

# Format:
# [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction,

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
Fixed K: 3
Best Test Size: 0.3
Best Accuracy: 0.6949152542372882
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

both K (neighbors) and test_size together

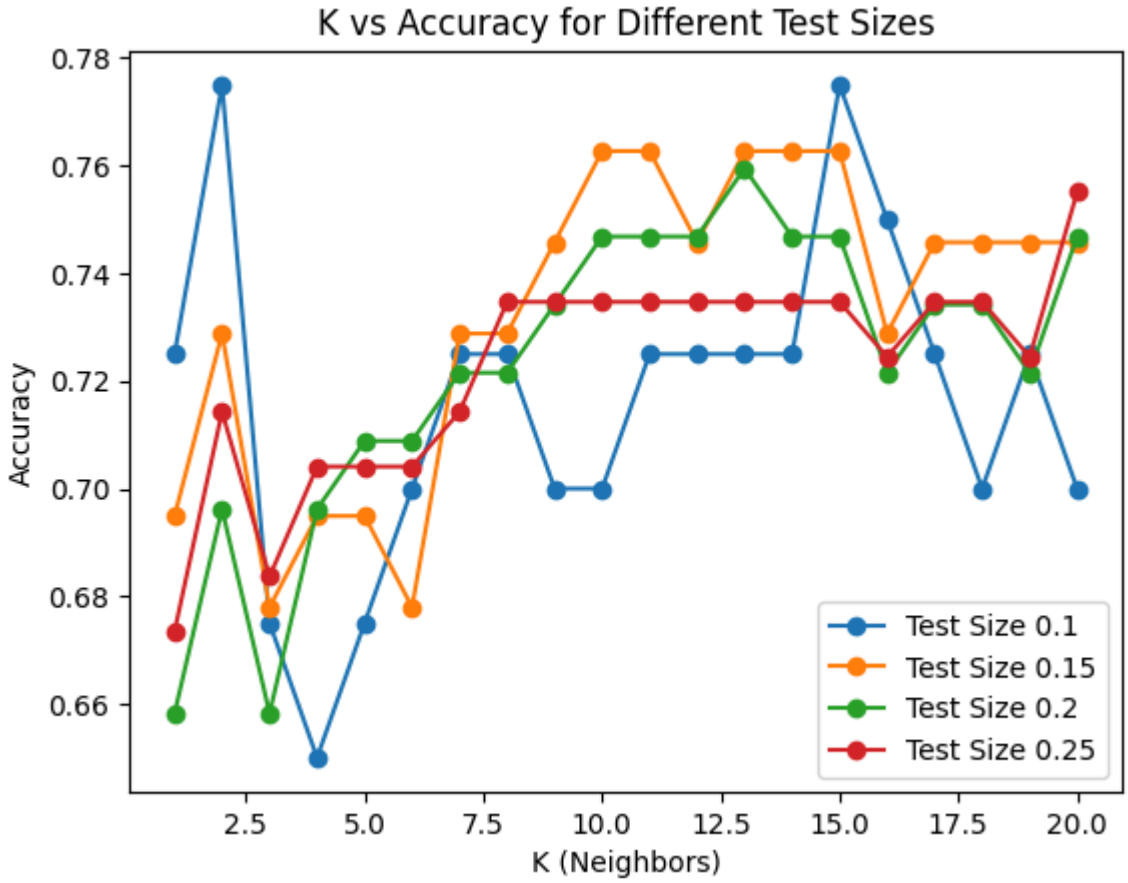
In [4]:

```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Define ranges
k_values = range(1, 21)
test_sizes = [0.1, 0.15, 0.2, 0.25]

results = []

best_accuracy = 0
best_k = None
best_test_size = None
best_model = None

# 6. Loop for both K and test_size
for ts in test_sizes:
    for k in k_values:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=ts, random_state=42
        )

        model = KNeighborsClassifier(n_neighbors=k)
        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)

        results.append((ts, k, acc))

        # Store best
        if acc > best_accuracy:
            best_accuracy = acc
            best_k = k
            best_test_size = ts
            best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
print("Best K:", best_k)
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 8. Convert results to DataFrame
results_df = pd.DataFrame(results, columns=['Test Size', 'K', 'Accuracy'])

# 9. Plot (for each test size)
plt.figure()
for ts in test_sizes:
    subset = results_df[results_df['Test Size'] == ts]
    plt.plot(subset['K'], subset['Accuracy'], marker='o', label=f'Test Size {ts}')

plt.xlabel("K (Neighbors)")
```

```
plt.ylabel("Accuracy")
plt.title("K vs Accuracy for Different Test Sizes")
plt.legend()
plt.show()

# 10. Predict using best model

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best K: 2
Best Test Size: 0.1
Best Accuracy: 0.775
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

Decision Tree Classifier example with exactly your requirements:

criterion = 'entropy'

test_size = 0.2

random_state = 42 (used everywhere possible)

using your diabetes.csv dataset

includes Accuracy, Precision, Recall, Specificity

plus prediction for one data point

In [5]:

```python
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Create model
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# 10. Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
specificity = TN / (TN + FP)

# 11. Output
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall (Sensitivity):", recall)
print("Specificity:", specificity)
print("Confusion Matrix:\n", cm)

# 12. Predict one new data point

# Format:
# [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction,

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = model.predict(new_data)

print("\nPrediction (0=No Diabetes, 1=Diabetes):", prediction[0])
```

```
Accuracy: 0.6962025316455697
Precision: 0.56
Recall (Sensitivity): 0.5185185185185185
Specificity: 0.7884615384615384
Confusion Matrix:
 [[41 11]
 [13 14]]

Prediction (0=No Diabetes, 1=Diabetes): 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

Try different max_depth values

Find best accuracy

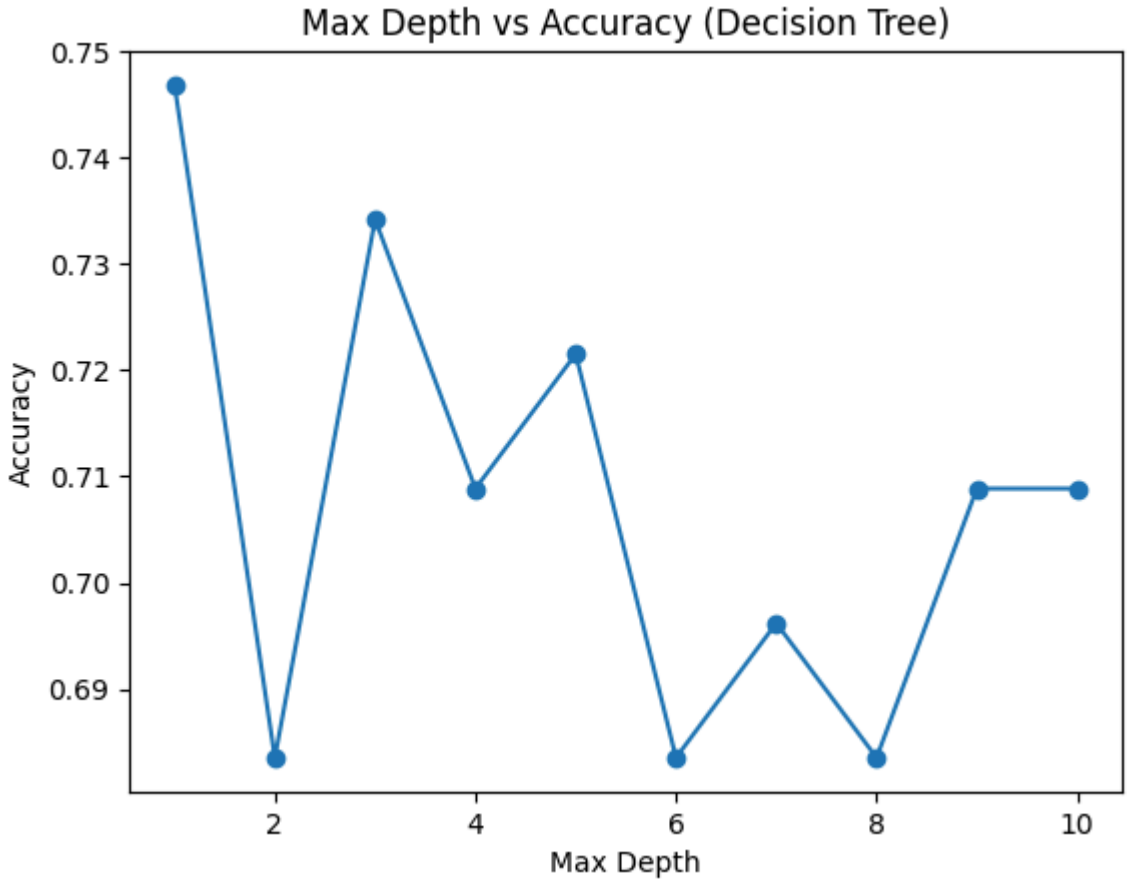
Plot Depth vs Accuracy

In [6]:

```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Try different max_depth values
depth_values = range(1, 11)

accuracy_list = []

best_accuracy = 0
best_depth = None
best_model = None

# 7. Loop for depth
for d in depth_values:
    model = DecisionTreeClassifier(
        criterion='entropy',
        max_depth=d,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```
    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)

    if acc > best_accuracy:
        best_accuracy = acc
        best_depth = d
        best_model = model

# 8. Print best result
print("===== BEST MODEL =====")
print("Best Max Depth:", best_depth)
print("Best Accuracy:", best_accuracy)

# 9. Plot Depth vs Accuracy
plt.figure()
plt.plot(depth_values, accuracy_list, marker='o')
plt.xlabel("Max Depth")
plt.ylabel("Accuracy")
plt.title("Max Depth vs Accuracy (Decision Tree)")
plt.show()

# 10. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best Max Depth: 1
Best Accuracy: 0.7468354430379747
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

max_depth + test_size together (with criterion='entropy' and random_state=42)

This will:

try multiple depths

try multiple test sizes

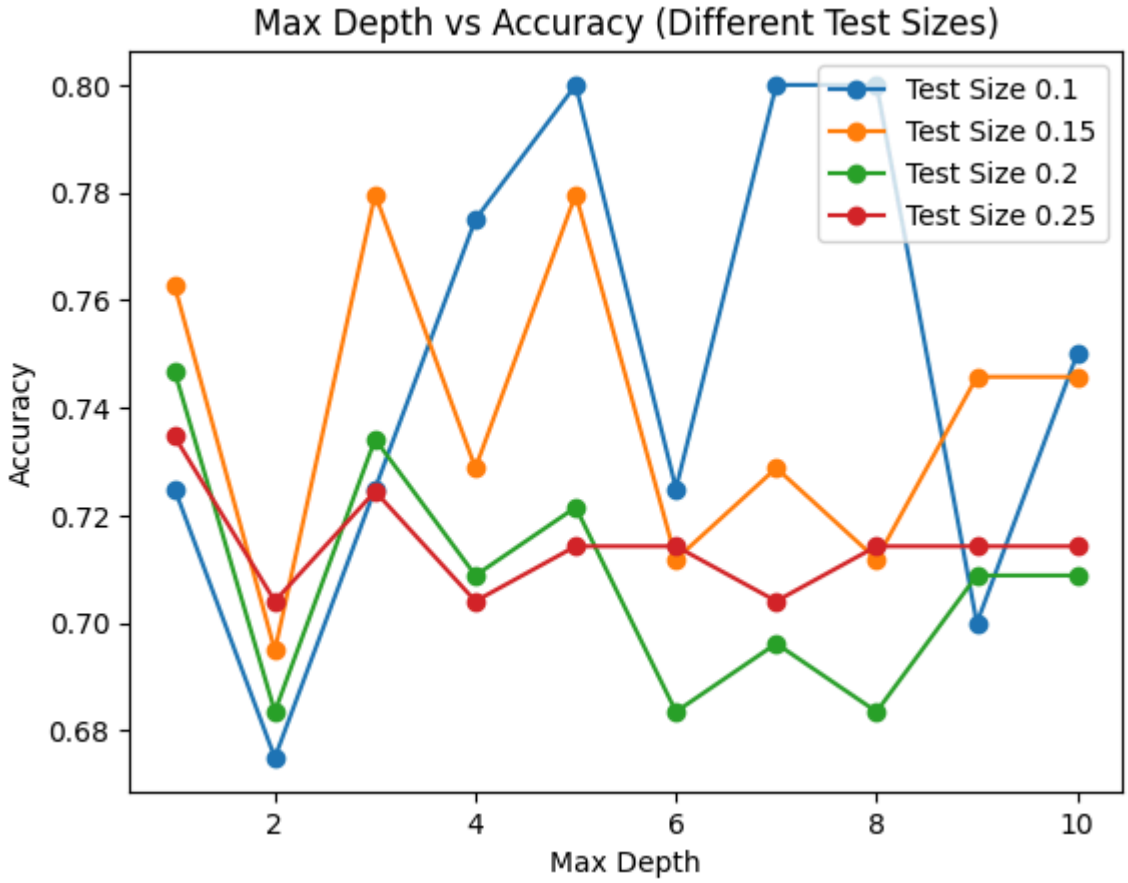
pick the best combination (highest accuracy)

plot results

In [8]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Parameters to test
depth_values = range(1, 11)
test_sizes = [0.1, 0.15, 0.2, 0.25]

results = []

best_accuracy = 0
best_depth = None
best_test_size = None
best_model = None

# 6. Loop for both parameters
for ts in test_sizes:
    for d in depth_values:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=ts, random_state=42
        )

        model = DecisionTreeClassifier(
            criterion='entropy',
            max_depth=d,
            random_state=42
        )

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
```

```
        results.append((ts, d, acc))

        # Store best model
        if acc > best_accuracy:
            best_accuracy = acc
            best_depth = d
            best_test_size = ts
            best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
print("Best Test Size:", best_test_size)
print("Best Max Depth:", best_depth)
print("Best Accuracy:", best_accuracy)

# 8. Convert results to DataFrame
results_df = pd.DataFrame(results, columns=['Test Size', 'Depth', 'Accuracy'])

# 9. Plot (for each test size)
plt.figure()
for ts in test_sizes:
    subset = results_df[results_df['Test Size'] == ts]
    plt.plot(subset['Depth'], subset['Accuracy'], marker='o', label=f'Test Size {ts}')

plt.xlabel("Max Depth")
plt.ylabel("Accuracy")
plt.title("Max Depth vs Accuracy (Different Test Sizes)")
plt.legend()
plt.show()

# 10. Predict one new data point using best model

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best Test Size: 0.1
Best Max Depth: 5
Best Accuracy: 0.8
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

Decision Tree

criterion = 'entropy'

max_depth = None (full tree, no limit)

try different test sizes only

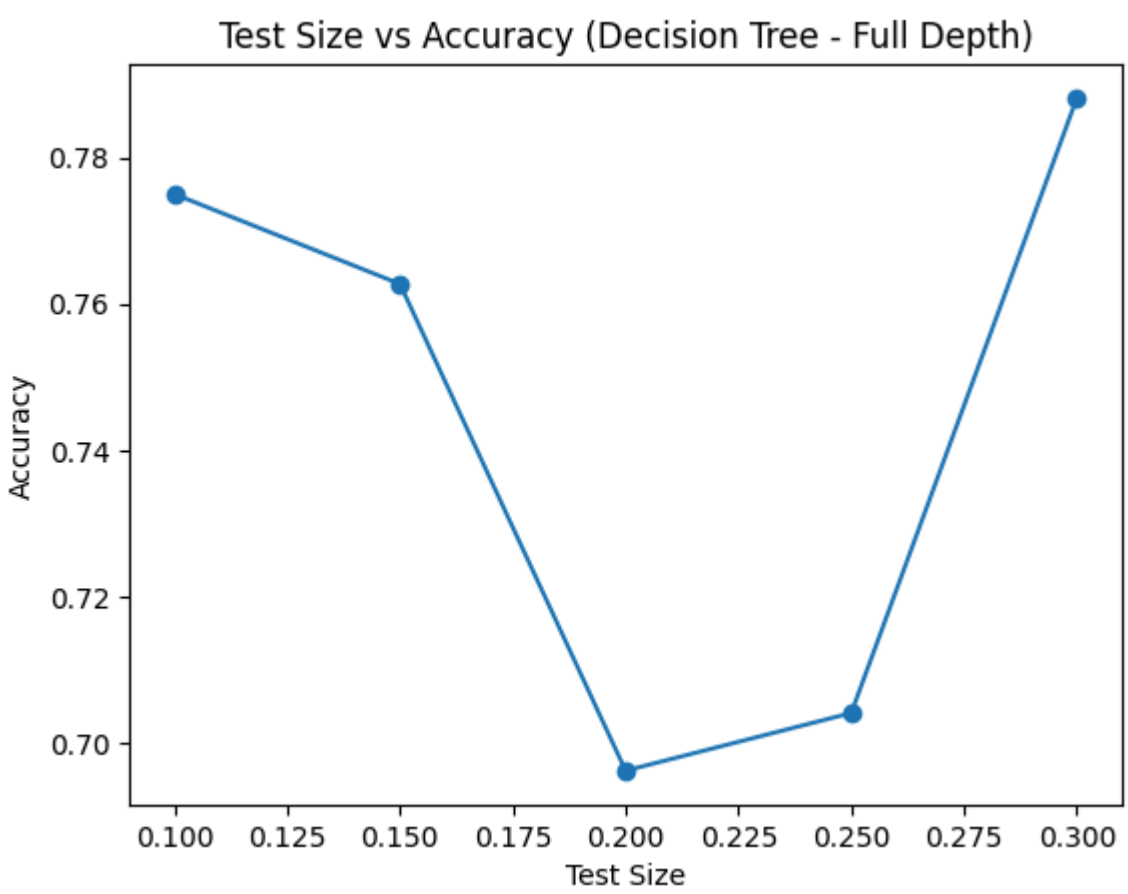
find best accuracy

plot results predict one value

In [9]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
```

```python
y = data['Outcome']

# 5. Different test sizes
test_sizes = [0.1, 0.15, 0.2, 0.25, 0.3]

accuracy_list = []

best_accuracy = 0
best_test_size = None
best_model = None

# 6. Loop through test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    model = DecisionTreeClassifier(
        criterion='entropy',
        max_depth=None,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)

    # Store best model
    if acc > best_accuracy:
        best_accuracy = acc
        best_test_size = ts
        best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
print("Max Depth: None")
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 8. Plot Test Size vs Accuracy
plt.figure()
plt.plot(test_sizes, accuracy_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("Accuracy")
plt.title("Test Size vs Accuracy (Decision Tree - Full Depth)")
plt.show()

# 9. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Max Depth: None
Best Test Size: 0.3
Best Accuracy: 0.788135593220339
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

Random Forest Classifier implementation exactly as you asked:

n_estimators = 51

criterion = 'entropy'

random_state = 42

test_size = 0.2

using your diabetes.csv dataset

includes Accuracy, Precision, Recall, Specificity

plus prediction for one data poin

In [10]:
```
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

```python
for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Create Random Forest model
model = RandomForestClassifier(
    n_estimators=51,
    criterion='entropy',
    random_state=42
)

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# 10. Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
specificity = TN / (TN + FP)

# 11. Output
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall (Sensitivity):", recall)
print("Specificity:", specificity)
print("Confusion Matrix:\n", cm)

# 12. Predict one new data point

# Format:
# [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction,

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = model.predict(new_data)

print("\nPrediction (0=No Diabetes, 1=Diabetes):", prediction[0])
```

```
Accuracy: 0.7468354430379747
Precision: 0.6666666666666666
Recall (Sensitivity): 0.5185185185185185
Specificity: 0.8653846153846154
Confusion Matrix:
 [[45  7]
 [13 14]]

Prediction (0=No Diabetes, 1=Diabetes): 0
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

try different n_estimators values

find best accuracy

plot Trees vs Accuracy keep:

criterion='entropy'

random_state=42

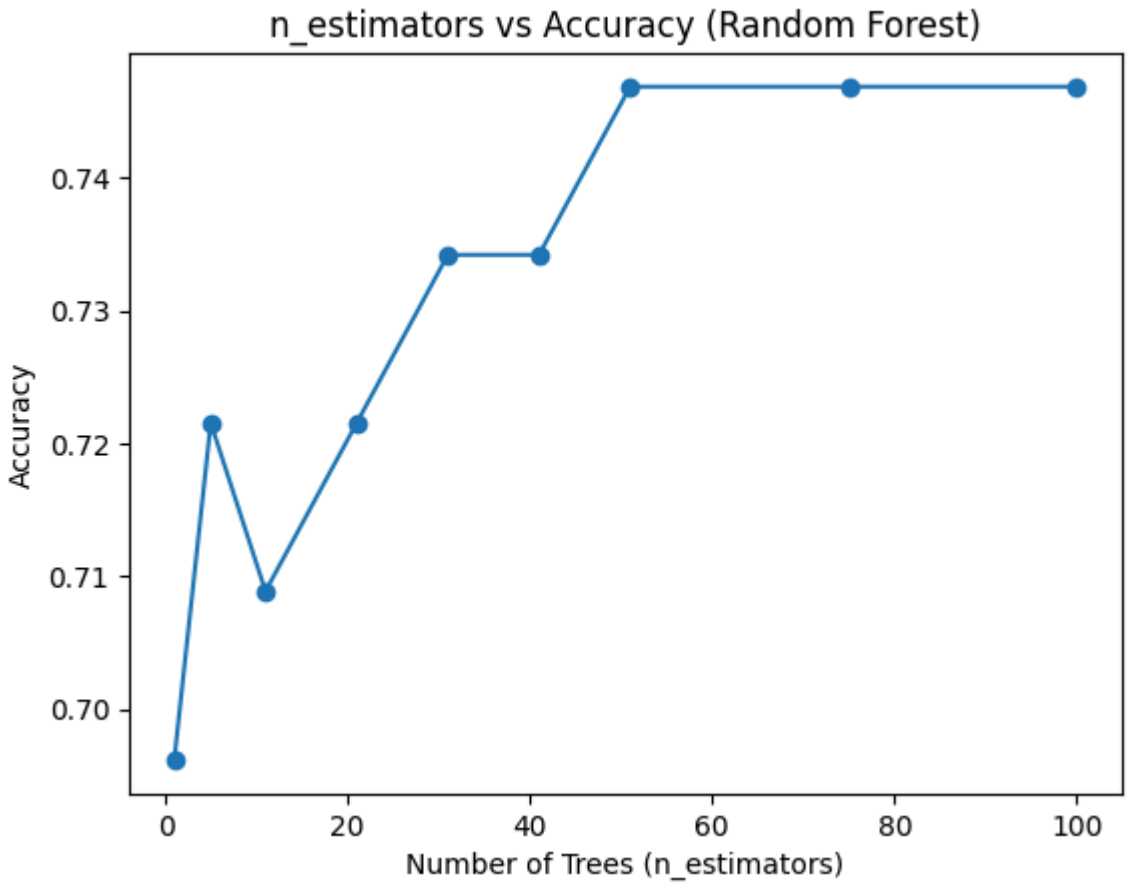
test_size=0.2

predict one data point using best model

In [11]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Try different number of trees
n_values = [1, 5, 11, 21, 31, 41, 51, 75, 100]

accuracy_list = []

best_accuracy = 0
best_n = None
best_model = None

# 7. Loop for n_estimators
for n in n_values:
    model = RandomForestClassifier(
        n_estimators=n,
        criterion='entropy',
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)
```

```
    if acc > best_accuracy:
        best_accuracy = acc
        best_n = n
        best_model = model

# 8. Print best result
print("===== BEST MODEL =====")
print("Best n_estimators:", best_n)
print("Best Accuracy:", best_accuracy)

# 9. Plot Trees vs Accuracy
plt.figure()
plt.plot(n_values, accuracy_list, marker='o')
plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Accuracy")
plt.title("n_estimators vs Accuracy (Random Forest)")
plt.show()

# 10. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best n_estimators: 51
Best Accuracy: 0.7468354430379747
```




```
Prediction using best model: 0
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

keep Random Forest fixed and test only different test_size values.

Fixed:

n_estimators = 51

criterion = 'entropy'

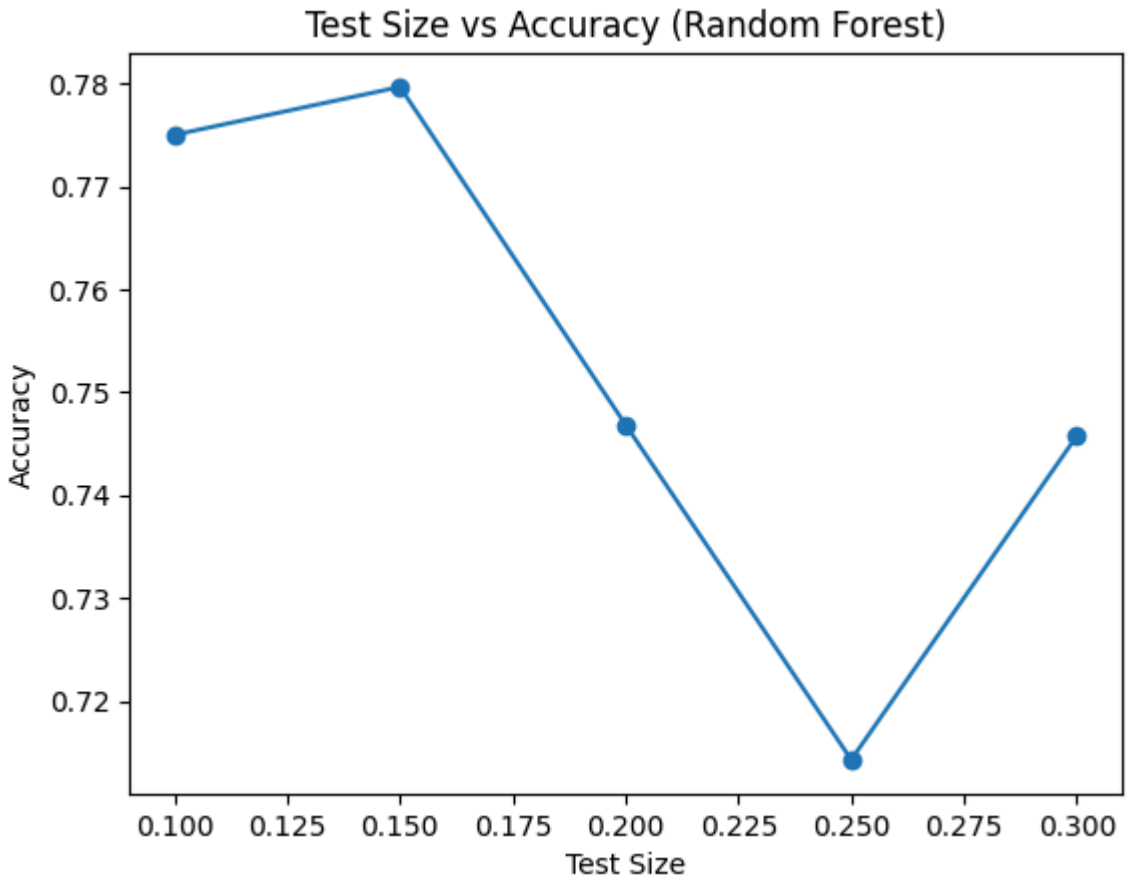
random_state = 42

Variable: test_size

In [12]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Different test sizes
test_sizes = [0.1, 0.15, 0.2, 0.25, 0.3]

accuracy_list = []

best_accuracy = 0
best_test_size = None
best_model = None

# 6. Loop for test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    model = RandomForestClassifier(
        n_estimators=51,
        criterion='entropy',
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)

    if acc > best_accuracy:
        best_accuracy = acc
        best_test_size = ts
        best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
```

```
print("n_estimators = 51")
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 8. Plot Test Size vs Accuracy
plt.figure()
plt.plot(test_sizes, accuracy_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("Accuracy")
plt.title("Test Size vs Accuracy (Random Forest)")
plt.show()

# 9. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
n_estimators = 51
Best Test Size: 0.15
Best Accuracy: 0.7796610169491526
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

combine n_estimators + test_size together

We will:

try multiple number of trees

try multiple test sizes

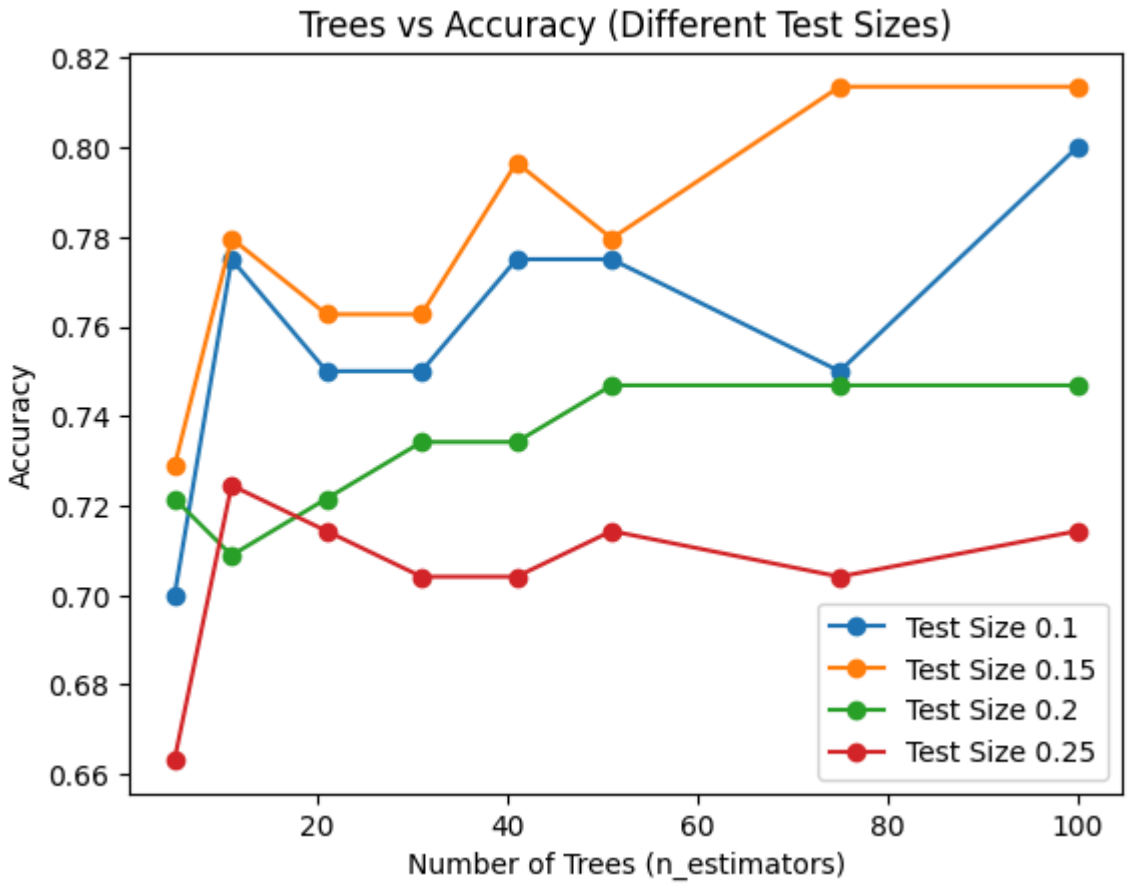
find best accuracy

plot results

In [13]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Define parameters
n_values = [5, 11, 21, 31, 41, 51, 75, 100]
test_sizes = [0.1, 0.15, 0.2, 0.25]

results = []

best_accuracy = 0
best_n = None
best_test_size = None
best_model = None

# 6. Loop for both parameters
for ts in test_sizes:
    for n in n_values:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=ts, random_state=42
        )

        model = RandomForestClassifier(
            n_estimators=n,
            criterion='entropy',
            random_state=42
        )

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        results.append((ts, n, acc))

        # Store best model
        if acc > best_accuracy:
            best_accuracy = acc
            best_n = n
            best_test_size = ts
            best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
print("Best n_estimators:", best_n)
print("Best Test Size:", best_test_size)
```

```
print("Best Accuracy:", best_accuracy)

# 8. Convert results to DataFrame
results_df = pd.DataFrame(results, columns=['Test Size', 'Trees', 'Accuracy'])

# 9. Plot (for each test size)
plt.figure()
for ts in test_sizes:
    subset = results_df[results_df['Test Size'] == ts]
    plt.plot(subset['Trees'], subset['Accuracy'], marker='o', label=f'Test Size {ts}')

plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Accuracy")
plt.title("Trees vs Accuracy (Different Test Sizes)")
plt.legend()
plt.show()

# 10. Predict using best model

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best n_estimators: 75
Best Test Size: 0.15
Best Accuracy: 0.8135593220338984
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

SVM (Support Vector Machine) classifier using your requirements:

kernel = 'rbf'

C = 1

test_size = 0.2

random_state = 42

dataset: diabetes.csv

includes Accuracy, Precision, Recall, Specificity

plus prediction for one data point

In [15]:
```python
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Create SVM model
model = SVC(
    kernel='rbf',
    C=1,
    random_state=42
)

# 7. Train model
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# 10. Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
specificity = TN / (TN + FP)

# 11. Output
print("Accuracy:", accuracy)
print("Precision:", precision)
```

```
print("Recall (Sensitivity):", recall)
print("Specificity:", specificity)
print("Confusion Matrix:\n", cm)

# 12. Predict one new data point

# Format:
# [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction,

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = model.predict(new_data)

print("\nPrediction (0=No Diabetes, 1=Diabetes):", prediction[0])
```

```
Accuracy: 0.7468354430379747
Precision: 0.7333333333333333
Recall (Sensitivity): 0.4074074074074074
Specificity: 0.9230769230769231
Confusion Matrix:
 [[48  4]
 [16 11]]

Prediction (0=No Diabetes, 1=Diabetes): 0
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
```

controls the trade-off between margin and misclassification

We will:

try different C values

find best accuracy

plot C vs Accuracy

keep:

kernel='rbf'

test_size=0.2

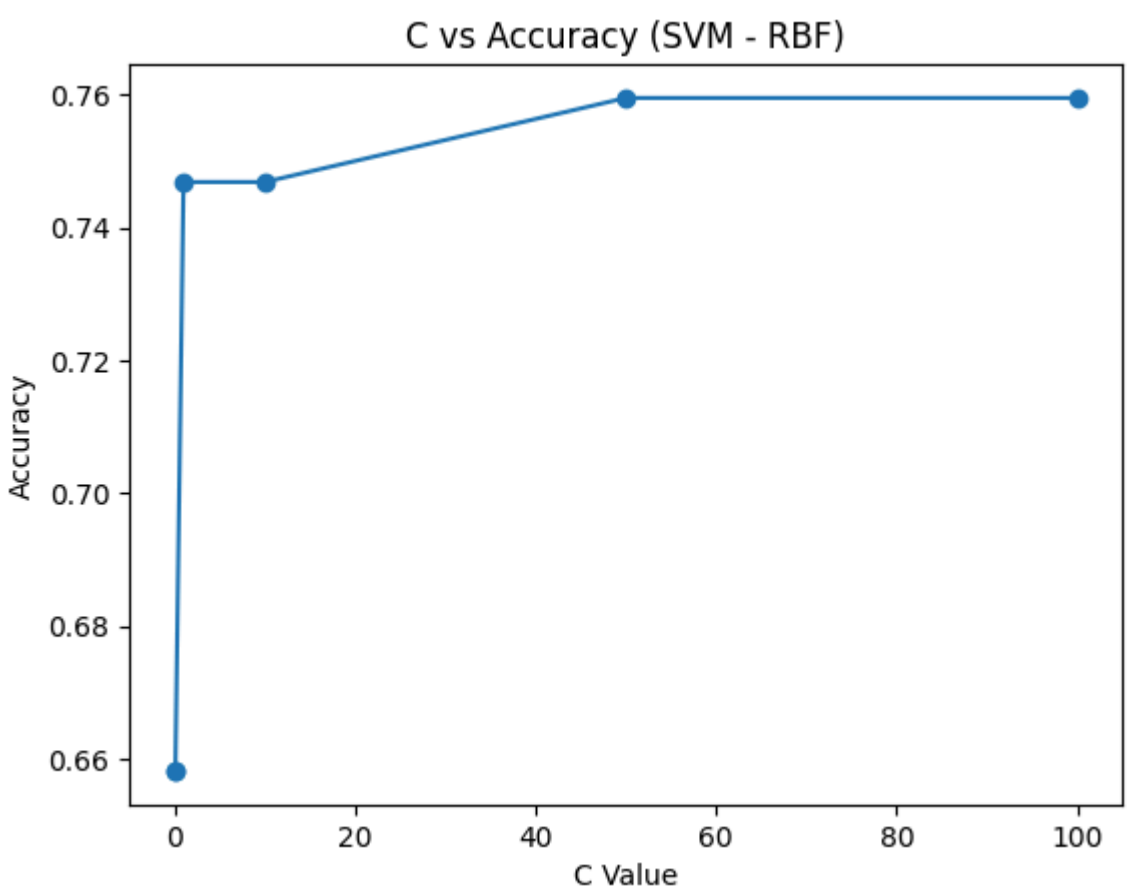
random_state=42

predict one value using best model

In [16]:
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

```python
for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Try different C values
c_values = [0.01, 0.1, 1, 10, 50, 100]

accuracy_list = []

best_accuracy = 0
best_c = None
best_model = None

# 7. Loop for C
for c in c_values:
    model = SVC(
        kernel='rbf',
        C=c,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)

    if acc > best_accuracy:
        best_accuracy = acc
        best_c = c
        best_model = model

# 8. Print best result
print("===== BEST MODEL =====")
print("Best C:", best_c)
print("Best Accuracy:", best_accuracy)

# 9. Plot C vs Accuracy
plt.figure()
plt.plot(c_values, accuracy_list, marker='o')
plt.xlabel("C Value")
plt.ylabel("Accuracy")
plt.title("C vs Accuracy (SVM - RBF)")
plt.show()

# 10. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best C: 50
Best Accuracy: 0.759493670886076
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
```

SVM (RBF kernel)

C fixed (e.g., C = 1)

try different test sizes

find best accuracy

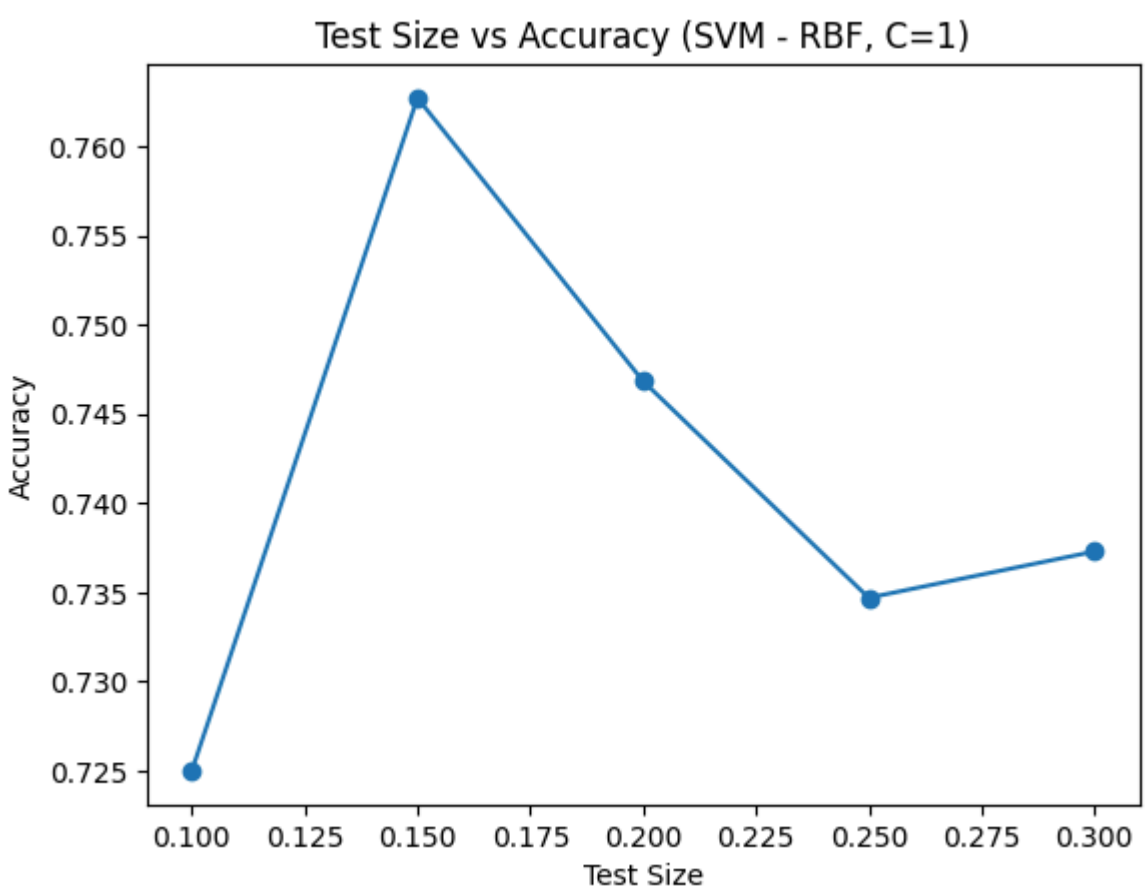
plot Test Size vs Accuracy

predict one value using best model

In [17]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
```

```
y = data['Outcome']

# 5. Fixed C value
c_value = 1

# 6. Different test sizes
test_sizes = [0.1, 0.15, 0.2, 0.25, 0.3]

accuracy_list = []

best_accuracy = 0
best_test_size = None
best_model = None

# 7. Loop through test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    model = SVC(
        kernel='rbf',
        C=c_value,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracy_list.append(acc)

    if acc > best_accuracy:
        best_accuracy = acc
        best_test_size = ts
        best_model = model

# 8. Print best result
print("===== BEST MODEL =====")
print("Fixed C:", c_value)
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 9. Plot Test Size vs Accuracy
plt.figure()
plt.plot(test_sizes, accuracy_list, marker='o')
plt.xlabel("Test Size")
plt.ylabel("Accuracy")
plt.title("Test Size vs Accuracy (SVM - RBF, C=1)")
plt.show()

# 10. Predict one new data point

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Fixed C: 1
Best Test Size: 0.15
Best Accuracy: 0.7627118644067796
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
```

combine C + test_size together (with kernel='rbf')

We will:

try multiple C values

try multiple test sizes

find best accuracy

plot results

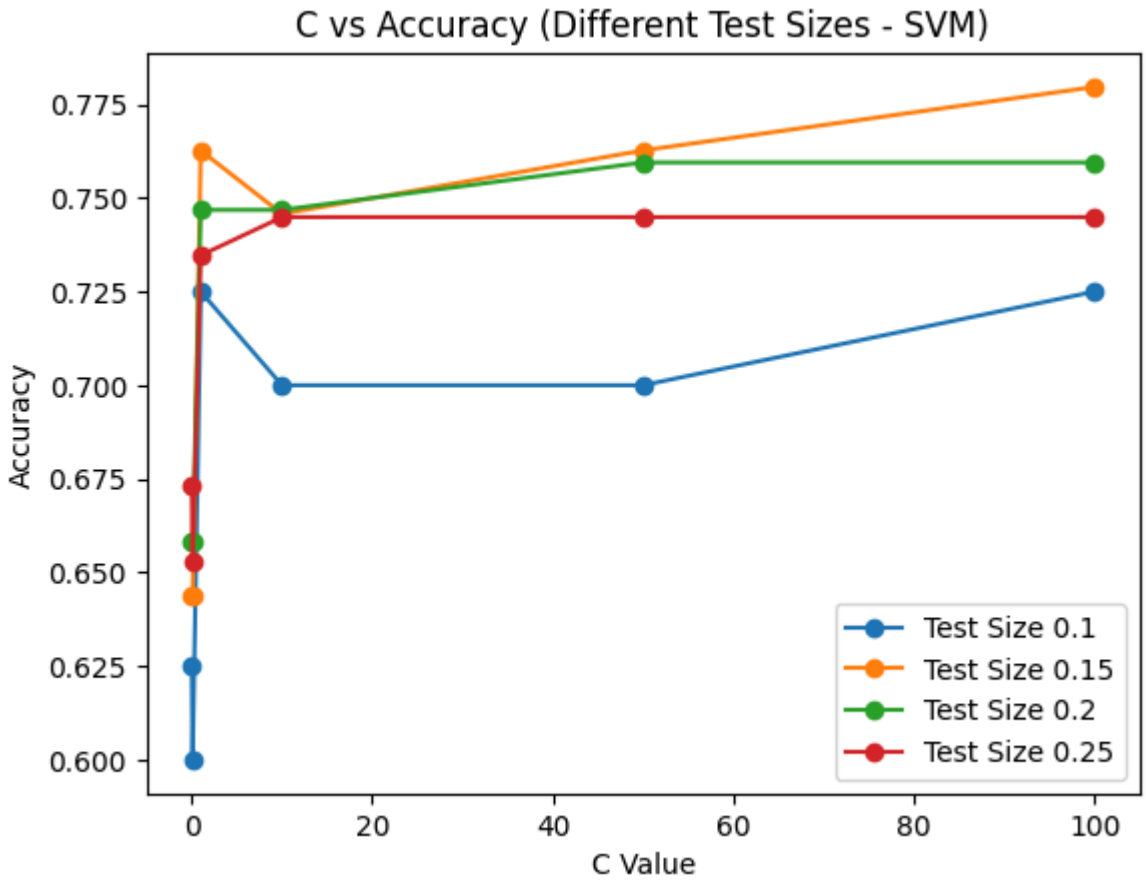
predict one value using best mode

In [19]:
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
```

```
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Define parameters
c_values = [0.01, 0.1, 1, 10, 50, 100]
test_sizes = [0.1, 0.15, 0.2, 0.25]

results = []

best_accuracy = 0
best_c = None
best_test_size = None
best_model = None

# 6. Loop for both parameters
for ts in test_sizes:
    for c in c_values:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=ts, random_state=42
        )

        model = SVC(
            kernel='rbf',
            C=c,
            random_state=42
        )

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        results.append((ts, c, acc))

        # Store best model
        if acc > best_accuracy:
            best_accuracy = acc
            best_c = c
            best_test_size = ts
            best_model = model

# 7. Print best result
print("===== BEST MODEL =====")
print("Best C:", best_c)
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)

# 8. Convert results to DataFrame
results_df = pd.DataFrame(results, columns=['Test Size', 'C', 'Accuracy'])

# 9. Plot (for each test size)
plt.figure()
for ts in test_sizes:
    subset = results_df[results_df['Test Size'] == ts]
    plt.plot(subset['C'], subset['Accuracy'], marker='o', label=f'Test Size {ts}')

plt.xlabel("C Value")
plt.ylabel("Accuracy")
plt.title("C vs Accuracy (Different Test Sizes - SVM)")
plt.legend()
plt.show()

# 10. Predict using best model
```

```
new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using best model:", prediction[0])
```

```
===== BEST MODEL =====
Best C: 100
Best Test Size: 0.15
Best Accuracy: 0.7796610169491526
```




```
Prediction using best model: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but SVC was fitted with feature names
  warnings.warn(
```

Compare KNN, Decision Tree, Random Forest, SVM

Try different test sizes

Store accuracy for each model

Plot Test Size vs Accuracy (for each model)

Show which model performs best overall

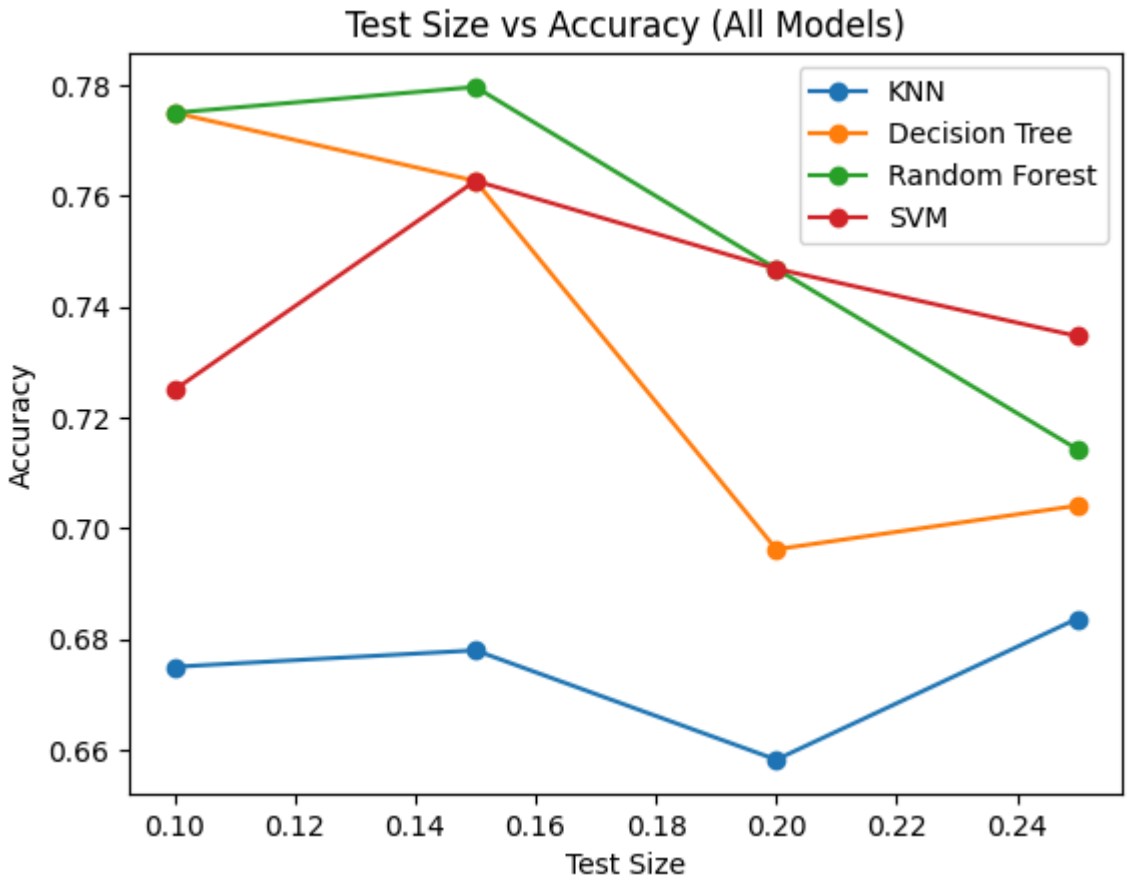
predict value

In [22]:
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```python
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Test sizes
test_sizes = [0.1, 0.15, 0.2, 0.25]

# 6. Store results
results = {
    "KNN": [],
    "Decision Tree": [],
    "Random Forest": [],
    "SVM": []
}

best_accuracy = 0
best_model_name = None
best_model = None
best_test_size = None

# 7. Loop through test sizes
for ts in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=ts, random_state=42
    )

    models = {
        "KNN": KNeighborsClassifier(n_neighbors=3),
        "Decision Tree": DecisionTreeClassifier(criterion='entropy', random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=51, criterion='entropy', random_
        "SVM": SVC(kernel='rbf', C=1, random_state=42)
    }

    # Train and evaluate
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)

        results[name].append(acc)

        # Track best model overall
        if acc > best_accuracy:
            best_accuracy = acc
            best_model_name = name
            best_model = model
            best_test_size = ts

# 8. Print best model
print("===== BEST MODEL OVERALL =====")
print("Model:", best_model_name)
print("Best Test Size:", best_test_size)
print("Best Accuracy:", best_accuracy)
```

```
# 9. Plot graph
plt.figure()

for model in results:
    plt.plot(test_sizes, results[model], marker='o', label=model)

plt.xlabel("Test Size")
plt.ylabel("Accuracy")
plt.title("Test Size vs Accuracy (All Models)")
plt.legend()
plt.show()

# 10. Predict using BEST model

new_data = [[2, 120, 70, 20, 80, 25.0, 0.5, 30]]

prediction = best_model.predict(new_data)

print("\nPrediction using BEST MODEL:", prediction[0])
```

```
===== BEST MODEL OVERALL =====
Model: Random Forest
Best Test Size: 0.15
Best Accuracy: 0.7796610169491526
```




```
Prediction using BEST MODEL: 0
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X d
oes not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

compare 4 algorithms on the same dataset:

KNN,Decision Tree,Random Forest,SVM

Using:

same train-test split (0.2, random_state=42)

same evaluation metric (Accuracy)

plus a comparison graph

In [20]:
```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 2. Load dataset
data = pd.read_csv('diabetes.csv')

# 3. Handle missing values (remove 0 rows)
cols_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols_with_zero:
    data = data[data[col] != 0]

# 4. Define X and y
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Initialize models
models = {
    "KNN (k=3)": KNeighborsClassifier(n_neighbors=3),
    "Decision Tree": DecisionTreeClassifier(criterion='entropy', random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=51, criterion='entropy', random_stat
    "SVM (RBF)": SVC(kernel='rbf', C=1, random_state=42)
}

# 7. Train and evaluate
accuracy_results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracy_results[name] = acc
    print(f"{name} Accuracy: {acc}")

# 8. Plot comparison
plt.figure()
plt.bar(accuracy_results.keys(), accuracy_results.values())
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Comparison (Diabetes Dataset)")
plt.xticks(rotation=30)
plt.show()
```

```
KNN (k=3) Accuracy: 0.6582278481012658
Decision Tree Accuracy: 0.6962025316455697
Random Forest Accuracy: 0.7468354430379747
SVM (RBF) Accuracy: 0.7468354430379747
```